Data Oblivious CPU: Microarchitectural Side-channel Leakage-Resilient Processor

Behnam Omidi
George Mason University
Fairfax, US
bomidi@gmu.edu

Ihsen Alouani
CSIT, Queen's University Belfast
Northern Ireland, UK
i.alouani@qub.ac.uk

Khaled N. Khasawneh
George Mason University
Fairfax, US
kkhasawn@gmu.edu

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11133149

***Abstract*—Mitigating microarchitectural side channels remains a central challenge in hardware security. Despite substantial research efforts, current defenses are often narrowly tailored to specific vulnerabilities, leaving systems exposed to a broader spectrum of microarchitectural side-channel attacks. In this paper, we propose a generic approach to mitigate side-channel attacks with minimal architectural changes. Unlike traditional approaches that focus on mitigating specific side channels, we propose a dynamic strategy that alters the decoding of the instructions into secure (side-channel resilient) or performance versions of the instructions, based on the data it is processing. Specifically, to minimize performance overhead, decoding to a secure version is selectively applied only when sensitive data are being processed, ensuring optimal performance for instructions operating on non-sensitive data.**

To evaluate our approach, we implement it on the RISC-V out-of-order BOOM processor. Our results demonstrate that the mechanism increases the utilization of FPGA resources by only 2% compared to the original design. Furthermore, it imposes 0% performance overhead for unprotected applications, while maintaining overhead between up to 25% for security-critical workloads. This work represents a scalable and efficient solution for defending against micro-architectural side-channel attacks without compromising system performance.

***Index Terms*—Microarchitectural side-channel attacks, Information Flow Tracking,**

I. Introduction

Modern processors are equipped with several optimization features to boost their performance. Although these features (micro-architectural components), namely cache, branch prediction, etc, are invisible on the architectural level, changes in their states can be observable later by side-channel attacks. For example, in cache side-channel attacks such as Flush+Reload [1], and Prime+Probe [2], the attacker can predict the value of the sensitive data by measuring the difference between the miss and hit time on data residing in the cache.

Although there are numerous works in the literature that aim to mitigate microarchitectural side-channel attacks [3]–[10], they either suffer from high-performance overhead or can prevent only one type of architectural side-channel attack and leave others open to exploit.

In this paper, we address this challenge by enabling the CPU to selectively bypass the use of micro-architectural components when executing instructions that process sensitive data, ensuring robust security without compromising performance for instructions that are processing non-sensitive data. To achieve this aim, the user is only required to annotate sensitive data at the application level. The operating system automatically maps these data to a sensitive location in memory. The processor tracks the propagation of sensitive data using information flow tracking. Lastly, the decoder decodes instructions to secure (bypasses microarchitectural components) or performance (uses micro-architectural components) based on the data the instructions are processing, whether the data are sensitive or not.

Furthermore, in this paper, we evaluate the security of our system against existing microarchitectural attacks [11], [12] and indicate that by annotating the secret in the program, the system blocks the creation of side channels based on microarchitectural states. We also analyze our proposal in terms of hardware area and performance. In terms of area, our proposal takes $< 2\%$ the area of the unmodified BOOM processor [13]. For performance overhead, we run the SPECspeed 2017 integer benchmark and modify some security-critical applications by annotating sensitive data. We show that the system has 0% performance overhead for unprotected applications and the overhead up to 25% for protected applications.

The key contributions of this paper are as follows.

- We introduced the Data Oblivious CPU design, a novel processor architecture that dynamically adapts instruction execution to mitigate microarchitectural side-channel attacks. Specifically, the proposed design employs a hybrid execution model, which decodes instructions into secure or performance versions based on the sensitivity of the data being processed.
- We provide a complete hardware prototype of our ideas, built on top of the RISC-V out-of-order BOOM processor, demonstrating the feasibility and practicality of our approach. All modifications in the BOOM architecture and software stacks are available online: https://github.com/beomidi/data_oblivious_cpu
- We evaluate the security, area, and performance of our methodology and show that it has minor overhead on hardware complexity, blocks micro-architectural side-channel attacks, while having acceptable performance overhead on protected applications (up to 25%), while having 0% performance overhead on unprotected applications.

II. Background and Related Work

A. Berkeley Out-of-Order Processor

RISC-V is an open-source, royalty-free ISA which follows the Reduced Instruction Set Computing (RISC) principle. It offers three privilege levels: machine mode, supervisor mode, and user mode [14] to protect different components of the software stack. Due to its simplicity and flexibility, it allows hardware developers to specify many optional extensions and leverage it in many application areas, ranging from small embedded systems to large-scale, high-performance servers.

The University of California, Berkeley, has been developing and maintaining the Berkeley Out-of-Order Machine (BOOM), an open-source synthesizable parameterized out-of-order RISC-V processor [13]. The architecture of BOOM consists of seven pipeline stages: Fetch, Decode/Rename, Rename/Dispatch, Issue/RegisterRead, Execute, Memory, and Writeback (Commit is not counted in the pipeline due to its asynchronous occurrence). It also implements sv39 page-based virtual memory system that supports 39-bit virtual address spaces. To achieve maximum utilization of the execution unit, the processor leverages several optimization features. The first and most important one is the implementation of out-of-order execution functionality, which allows the processor not to wait for independent instructions to complete their execution in sequential order. It also has two levels of branch prediction- a fast Next-Line Predictor (a kind of Branch Target Buffer) and a slower but more complex Backing Predictor (a complicated structure like a GShare predictor). Moreover, it supports a private non-blocking cache (Hellacache) which is designed for in-order processors and is connected by a $shim$ to the BOOM. Correspondingly, the BOOM has all the convenient characteristics that can be exploited to establish microarchitectural attacks.

B. Micro-architectural Attacks

Micro-architectural attacks compromise the security of the system by monitoring the micro-architectural effects of specific optimization through side channels including cache access pattern [7], [15], [16], branch access patterns [17], faults [18], functional unit timing characteristics [19], power consumption [20], [21] and electromagnetic effects [22]. The attacker chooses an appropriate side-channel based on the characterization of the micro-architectural component. These characterizations can be categorized as state-less, state-full, private, or public [6]. State-less resources are resources such as functional units that don't maintain the state of executed instructions. However, State-full resources like caches and branch predictions keep the state of instruction. Private resources like private L1 cache refer to those that cannot be accessed from other cores. In contrast, public resources, such as the last-level shared cache, are shared among every core.

C. Information Flow Tracking

Several efforts have been made to track the information in the system and mitigate the side-channel attacks. They can be categorized as software-based and hardware-based solutions. Software-based mechanisms attempt to change the way of programming [8], [23] or analyze the code statically to insert some blocking instructions [24], [25] by considering the existing potential side channels. Although their solutions can alleviate some types of attack, they impose a large performance overhead on the system. Moreover, since the microarchitectural components are invisible to the developer, it is impossible to secure the application against all micro-architectural attacks at the software level. However, multiple hardware information flow tracking techniques have been described at various hardware levels. Taram *et al.* [10] propose a mechanism to dynamically alter the decoding of the instruction stream and inject new micro-ops (fences) under specific conditions. They mark any information from user address space IO devices as confidential information, which imposes so much overtainting space on the system, causing a large performance loss. Furthermore, the proposed micro-ops are memory-ordered instructions that can only prevent microarchitectural attacks on the memory hierarchy system. Yu *et al.* [26] attempt to extend the Instruction Set Architecture (ISA) of the processor to support data oblivious programming to be resilient against microarchitectural side-channels. Since they only consider oblivious program vulnerabilities, their implementation terminates the program when it violates a specific condition, making it difficult to debug the program. In addition, they consider part of the memory as storage of confidential information metadata, which increases the size of the application. Schwarz *et al.* [9] proposed a mechanism to alleviate transient execution attacks by tracking information from the application to the hardware. Although they discussed how to track the secret in the hardware, their evaluation is a software-based over-approximation and only considers the cache hierarchy as a source of the leakage. Sabbagh *et al.* [27] introduces an information flow tracking mechanism on the RISC-V architecture to activate the data memory fencing on the secret. Their tracking technique only considers speculative execution behavior and defends against cache-side channels.

III. Threat Model

We consider a scenario where a victim program operates on a shared computing platform alongside potentially adversarial software. The adversary's objective is to exploit microarchitectural side channels to extract private information from the victim program. This private information may include confidential inputs provided by external parties or sensitive internal program states, such as cryptographic keys. The victim program itself is assumed to be public.

We consider contention-based attacks [6], [28], fault injection attacks (for example, DRAM attacks [29], [30]), EM radiation [22], and power side-channel attacks [20] to be out of the scope of our study.

IV. Methodology

In this section, we present the design of Data Oblivious CPU, a microarchitectural side-channel leakage-resilient processor. The key idea behind the Data Oblivious CPU is to dy-

namically adapt the decoding of instructions based on the sensitivity of the data being processed. Specifically, we introduce architectural mechanisms to classify and track sensitive data, selectively switching between secure (side-channel-resilient) and high-performance modes of execution. The secure version of instructions ensures that microarchitectural components do not update or change their state based on sensitive data, effectively eliminating the risk of side-channel leakage. At the same time, high-performance execution is maintained for operations involving non-sensitive data, ensuring minimal impact on overall system performance.

Data Oblivious CPU employs a multi-level defense strategy consisting of the following components:

1) Sensitive memory mapping
2) Sensitive data tracking
3) Dynamic decoding
4) Data-dependent behavior elimination
5) Software support for the hardware feature

A. Sensitive Memory Mapping

In modern processors, there are plenty of reserved bits in the page table entries that are embedded for future use. Figure 1 illustrates the RISC-V architecture page table entry (sv39). Sv39 supports a 39-bit virtual address space divided into 4 KB pages. Each page table entry has 64 bits, including a 44-bit physical address field and several flags. There are 10 bits between bits 54 to 64 that are considered reserved bits, which can be used for the physical page number extension. Since it is always 0, repurposing the last reserved bit as a *Sensitive (S)* bit does not have any side effect on the system.

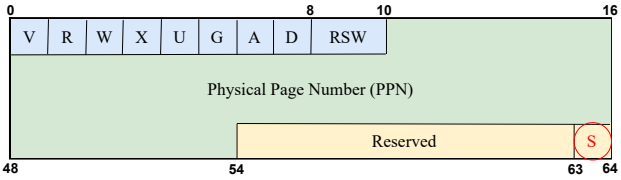

Fig. 1: RISC-V page table entry

B. Sensitive Data Tracking

Sensitive memory mapping ensures that sensitive data in memory are identified. However, to fully secure the system, it is necessary to extend this protection to registers that hold sensitive values. Registers in existing CPUs lack mechanisms for tracking the confidentiality of their content, requiring hardware modifications to implement this functionality. Inspired by prior work on hardware tainting mechanisms, we propose a simple yet effective tainting system to track and protect sensitive data in registers.

Specifically, we introduce an additional taint bit for each register. This bit indicates whether the value stored in the register is sensitive (if the bit is set). The taint propagates dynamically through the processor pipeline as follows: (a) *Memory to Register*: When a load instruction accesses sensitive memory, the taint bit of the destination register is set; (b) *Register to Register*: Operations involving one or more tainted registers, as source operands, result in tainting the destination register. This ensures that results derived from sensitive data are also treated as sensitive.

Registers are untainted in specific scenarios to prevent over-tainting, simplifying the tracking mechanism, and improving performance. Untainting occurs when a register's content is fully replaced with an immediate value or data from an untainted memory location, or when a tainted register is written to non-sensitive memory. This assumes that such writes are either intentional actions by the developer or program bugs to be addressed. For example, the output of a cryptographic cipher may not require continued protection, allowing the register to be untainted after the operation.

Additionally, taint information must be preserved across context switches and interrupts to ensure consistent tracking. We propose a dedicated Control Status Register (CSR) to store the taint state of all registers. During a context switch or interrupt, the operating system saves this register alongside other processor state information. Upon returning from the interrupt or context switch, the taint values are restored to maintain the integrity of the tracking system.

Lastly, since taint information is required at the decoding stage, we ensure accurate taint propagation by recovering the taint state from branch mispredictions by rolling back to the last committed taint state before the mispredicted branch. This can be achieved by encoding the speculative taint state into the reorder buffer as an additional state bit for each entry. When instructions are flushed due to a misprediction, the corresponding speculative taint state is also flushed. The speculative taint state is only moved to the committed taint state in the register file at the commit stage of the pipeline when the instruction is retired.

C. Dynamic decoding

A key innovation in the proposed architecture is its dynamic decoding mechanism, which adapts instruction execution based on the sensitivity of the data being processed. At the decoder stage, instructions are evaluated for their processing of tainted (sensitive) data. If an instruction involves tainted operands or operates on sensitive memory regions, it is dynamically translated into a secure version to ensure side-channel resilience. Conversely, instructions involving untainted data are translated into performance-optimized versions for maximum efficiency.

D. Data-dependent behavior elimination

To safeguard the system against side-channel attacks, it is essential to eliminate any observable microarchitectural footprint influenced by execution based on sensitive data. Therefore, to achieve broad compatibility and efficiency, the instruction set architecture (ISA) should support a range of instruction types designed for data-oblivious execution while minimizing hardware changes. This includes:

- Arithmetic Operations: All integer and floating-point arithmetic instructions should have a constant-time be-

havior version if they only have a data-dependent behavior version.

- Memory operations: The load and store instructions should have a secure version optimized for confidential addresses to implement oblivious memory access. For example, against cache side-channel attacks, all memory instructions accessing sensitive regions must bypass the cache hierarchy, ensuring that they leave no traceable cache or TLB state changes.
- Control Flow Operations: Control flow instructions should include a secure version for sensitive operands, ensuring that sensitive control flow decisions are executed securely and do not leave observable traces in microarchitectural components, enabling robust protection against side-channel leakage. For example, for the branch target buffer (BTB) and return stack buffer (RSB), the processor should not modify these components when executing instructions that modify their states, such as jumps, branches, and returns.

E. Software Support

To support the proposed hardware modifications, changes are required at the application, compiler, and operating system levels. Developers must annotate sensitive information in their applications, marking data that require confidentiality. The compiler processes these annotations, placing the identified sensitive data into a dedicated section within the program binary. The operating system is then responsible for mapping this section to sensitive memory pages, ensuring secure handling of the data.

Additionally, the operating system manages the preservation and restoration of register taint values during context switches and interrupts.

V. Implementation

In this section, we present the implementation of our mechanism on the BOOM processor to make it resilient against microarchitectural side-channel attacks. Since the method is a multi-level countermeasure, we first propose all modifications made to the software and then discuss hardware extensions to support the security of the processor.

A. Software

We propose a runtime library and kernel module and modify the compiler and operating system to follow the secret in the software. This section introduces all changes in the software in more detail.

Runtime Library: We develop a run-time library that can set confidential data in the sensitive memory region via a kernel module. The user is just required to include the library in their application and annotate the secret with the $nouarch$ keyword. The keyword guides the linker to allocate a secure section for the variable in the binary. In addition, the library has a constructor function that initializes the run-time requirement and only activates on application startup. To dynamically mark sensitive data, the library provides $malloc\_nouarch()$ and $free\_nouarch()$ functions that immediately allocate and release the heap memory as secure.

```
_save_context:
...
csrr s6, CSR_TAINT //taintCSR -> s6
REG_S s6, PT_TAINT(sp) //s6 -> stack
....

restore_all:
...
REG_L a2, PT_TAINT(sp) stack -> a2
csrw CSR_TAINT, a2 // a2 -> taintCSR
...
sret
```

Listing 1: (Pseudo-)assembly for keeping the taint CSR during a context switch

Compiler: In section IV-E, we mentioned that the compiler is responsible for creating a secure section in the binary. We exploit the existing attribute $\_\_attribute\_\_((annotate(\text{"}secret\text{"})))$ to dedicate a specific section in the elf file. Therefore, all $nouarch$ keywords are replaced by the secret attribute, and the compiler automatically allocates a secret section in the binary.

Page-Table Entry: For backward and future compatibility, we repurposed bit 64 in the RISC-V page-table entry, which is a reserved bit to indicate the secret pages. By setting the bit, the page is considered a secret region in memory mapping.

Kernel Module: We develop some operating system modifications as a kernel module. The module is in charge of mapping the secure section in binary to the sensitive memory pages. It prepares an interface to the runtime library to change the corresponding bit in the entry (bit 64). The corresponding TLB entry and the cache are flushed subsequently to make sure that the sensitive data do not change the micro-architectural states.

Operating system: The operating system keeps and reloads the content of each register taint on the context switch and interrupts. The pseudo-code 1 illustrates the operating system saving and restoring process of the CSR register to/in the stack during the context switch. We spill the taint CSR value to the register taint value later in the hardware on the return instructions that occur at all interrupt routines. Therefore, in this process, we are not concerned with the overwriting of a register with an immediate value, which untaints the value of the register taint.

B. Hardware

We modify the TLB, registers, microarchitectural components, and CSRs to support secret tracking on the hardware level.

Translation Lookaside Buffer: Since the PTEs are cached in TLB, we modify the TLB to cache the secure bit while bringing the entry. Each memory request should translate

its virtual address to the effective address. Therefore, in the BOOM processor, upon calculation of the effective address, the secure bit propagates to the corresponding taint bit in the register. Since this operation (effective address calculation) happens in the execution and we need the information of the taint in decode time, the taint and the instruction are stored in a temporary register and the pipeline would be flushed.

Registers: To track the taint, all physical registers in the architecture are extended by a tain bit. Therefore, in the operation, if one of the physical source registers of the executing instruction is tainted, we propagate the taint value to the destination physical register. During the commit stage, the taint of the physical register is copied to the taint of the committed register. Moreover, the processor monitors all the immediate loads and insensitive memory requests to untaint the corresponding registers.

Micro-architectural components: All memory requests in BOOM pass through the Hellacache (a private cache in the architecture). Therefore, we modify this cache to uncache the sensitive data. Moreover, the processor flushes the BTB and RSB when decoding an instruction having a secret operand to clear the possible changes in the components. For instructions carrying sensitive information, the processor executes them in order by keeping them in the issue stages until reaching the head of the Reorder Buffer.

Control Status Register: As we discussed in section IV-B, tracking the information requires a structure in hardware to keep the status of taints during interrupts and context switches. Hence, we add a new CSR called taintCSR, which stores/restores the taint status on an interrupt (consider the context switch as a type of interrupt). We set the address of the register in the supervisor space (0x190) to be only accessible by the operating system. When one of the interrupt bits is set in the machine mode status register (mstatus), which is a CSR to track the status of the hardware thread's current operating state, all register taint values are loaded into taintCSR to be stored in the memory structure by the operating system. To track back and restore the taint values, the processor checks $sret$ and $mret$ instructions, which execute at the end of each interrupt routine.

VI. Evaluation

In this section, we discuss the security and performance evaluation of the proposed method. We modify the BOOM processor to evaluate our technique. The processor configuration has been shown in the table I. We used Ubuntu 22.04 LTS with modified kernel version 5.19.0 to run on BOOM. The Xilinx KCU105 evaluation board has been leveraged to evaluate our processor. To this end, we modify the configuration of the chipyard, an open-source integrated SoC design and simulation framework, to make it compatible with the existing board.

To compare the compute-intensive performance of the system, we run the Spec2017 benchmark on the original BOOM and the modified one. As we expected, the results demonstrate the 0% performance overhead since we do not annotate any data as secret and the system does not block any microarchitectural components. We use security-critical applications in [9] to measure the performance of our work by annotating the secret values on them.

Parameters	Value
CPU	1 core RISC-V out-of-order sonicBOOM
Core	32 ROB entries, 8 TLB entries, 8 Load Queue entries, 8 Store Queue entries, 52 physical registers, 48 floating point registers, 4 fetch size
L1-I Cache	2KB, 64-set, 4-way
L1-D Cache	2KB, 64-set, 4-way

TABLE I: Configuration of the BOOM

OpenSSL: We evaluated the performance overhead of our technique by encrypting a message using OpenSSL's chacha, and rc4 ciphers. By notating the key using the secure attribute, we protect the key during execution in the system.

OpenSSH: OpenSSH is the premier connectivity tool for remote login with the SSH protocol, consisting of remote operation tools like ssh, scp, sftp. The confidential information in these operations is the private key, which is stored in memory and is susceptible to micro-architectural attacks. Hence, to evaluate the performance of our system, we annotate the private key in the application and measure the time it takes to connect through the ssh or copy the file through scp.

OATH Tool: Open AuTHentication (OATH) one-time password tool is designed to generate and validate one-time passwords. The tool calculates a cryptographic hash over the shared secret and the current time using a shared key between the user and the service. To evaluate the performance of the application, we annotate the shared secret and generate one-time passwords.

Lastpass-cli: The LastPass command-line application is an open-source application that allows users to create, edit, and retrieve passwords. Connects to a remote server and the user needs to provide a master password to access the data. To enhance the security of the tool, we annotate buffers that store the master password and the decryption key with the secure attribute. We repeatedly queried a password to measure the performance overhead of the technique on the application.

VeraCrypt: VeraCrypt is a free open-source disk encryption software for Windows, Mac OSX, and Linux. To store confidential information, VeraCrypt uses the SecureBuffer class. Hence, we mark all SecureBuffer instantiations in the program to protect the application. We analyze the performance of the application by mounting and encrypting the data with a password and key.

NGINX: Engine x is an HTTP web server that can be used for a variety of web server tasks, including Web serving, Load balancing, and reverse proxying, etc. It can also be used as an HTTPS server. Therefore, we modified NGINX to protect the certificate key in HTTPS communication. We use siege load testing and a benchmarking utility to put the server under stress testing to evaluate its performance. We simulate 255 clients for

300 seconds with siege. With this stress test, the number of transactions has decreased from 59 to 56, which means 5.08% performance overhead.

Figure 2 shows the performance overhead for the rest of the security-critical applications. The results indicate increasing the execution time between 1.33% for the *chacha* to 25.63% for the *ssh*

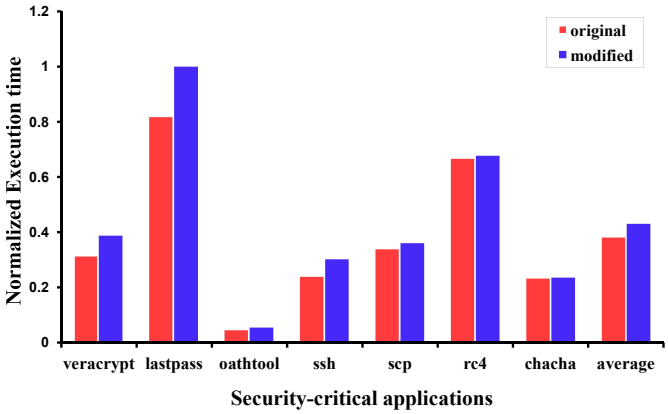

Fig. 2: Performance overhead for security-critical applications

To measure the area overhead of the proposed method, we generate the post-synthesis report of the BOOM processor on the Kintex UltraScale XCKU040-2FFVA1156E device. The different utilizations of the original and modified BOOM resources are shown in Table II, which indicates an increase of 0% to 3% for the modified BOOM resources.

Resources	Original (#)	Modified (#)	Overhead (%)
LUT1	1025	1004	-2.04
LUT2	6268	6127	-2.24
LUT3	14602	15053	+3.08
LUT4	15105	14484	-0.41
LUT5	18155	18403	+1.36
LUT6	65103	65980	+1.34
MUXF7	11300	11576	+2.44
MUXF8	2535	2543	+0.31
FDRE	76841	78201	+1.76
FDSE	1236	1229	-0.56
Total	211145	213596	1.16

TABLE II: Resource utilization of the original BOOM and modified one on KCU105 FPGA (LUT# = # inputs Lookup Table, MUXF# = # inputs multiplexer, FDRE = D-type flip-flop with clock enable and synchronous reset, FDSE= D-type flip-flop with clock enable and synchronous set)

To evaluate the security of the system, we execute all existing microarchitecture side-channel attacks [11], [12] on RISC-V architecture. The system equipped with our mechanism has not shown any leakage in any of them. Figure 3 demonstrates the impact of our approach on cache timing side-channel attacks (one of the common exploitable attacks) represented in pseudo-code 2. The program was written to measure the hit-and-miss rates of a specific address. The experiment shows that the timing for hit and miss is distinguishable for the original system, while the modified one maps both times to a single point.

```
char nouarch *address;

for(int i=0; i<num_of_measurements; i++){
    hit = measure_access_time(address);
    hit_histogram[hit]++;
}

for(int i=0; i<num_of_measurements; i++){
    //Flush the cache in original system
    //Remove the line for modified system
    flushCache(address, sizeof(address))
    miss = measure_access_time(address);
    miss_histogram[miss]++;
}
...
```

Listing 2: Pseudo-code of cache timing side-channel attack

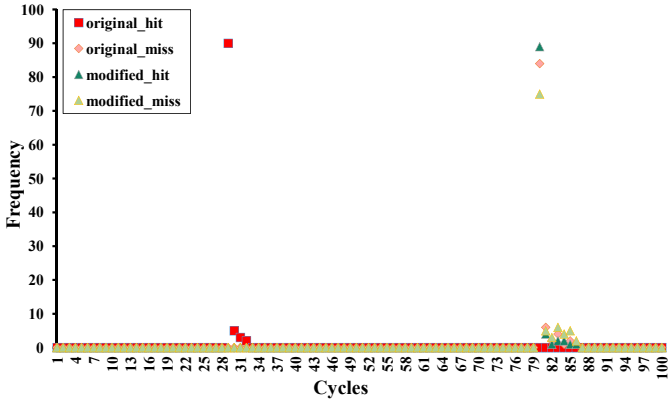

Fig. 3: Cache timing side-channel attack effect on system

VII. Conclusion

In this paper, we introduce a technique to mitigate microarchitectural side-channel attacks on state-of-the-art processors. We modify all levels of abstraction on the BOOM processor, including the application, compiler, operating systems, and hardware, to track sensitive information and eliminate all the footprints leveraged by these attacks. We evaluate our technique on commonly used microarchitectural attacks and some security-critical applications. Our results illustrate that our method can block side-channel attacks and has 0% performance overhead in unprotected applications and up to 25% overhead on average in protected applications.

VIII. Acknowledgment

The work in this paper is partially supported by the National Science Foundation grant CNS-2155002.

References

[1] Y. Yarom and K. Falkner, "{FLUSH+ RELOAD}: A high resolution, low noise, l3 cache {Side-Channel} attack," in *23rd USENIX security symposium (USENIX security 14)*, 2014, pp. 719–732.

[2] C. Trippel, D. Lustig, and M. Martonosi, "Meltdownprime and spectreprime: Automatically-synthesized attacks exploiting invalidation-based coherence protocols," *arXiv preprint arXiv:1802.03802*, 2018.

[3] M. Kayaalp, K. N. Khasawneh, H. A. Esfeden, J. Elwell, N. Abu-Ghazaleh, D. Ponomarev, and A. Jaleel, "Ric: Relaxed inclusion caches for mitigating llc side-channel attacks," in *Proceedings of the 54th Annual Design Automation Conference 2017*, 2017, pp. 1–6.

[4] J. Bahrami, V. B. Dang, A. Abdulgadir, K. N. Khasawneh, J.-P. Kaps, and K. Gaj, "Lightweight implementation of the lowmc block cipher protected against side-channel attacks," in *Proceedings of the 4th ACM Workshop on Attacks and Solutions in Hardware Security*, 2020, pp. 45–56.

[5] A. Dhavlle, S. Rafatirad, K. Khasawneh, H. Homayoun, and S. M. P. Dinakarrao, "Imitating functional operations for mitigating side-channel leakage," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 4, pp. 868–881, 2021.

[6] N. Nazari, B. Omidi, C. Fang, H. M. Makrani, S. Rafatirad, A. Sasan, H. Homayoun, and K. N. Khasawneh, "Specscope: Automating discovery of exploitable spectre gadgets on black-box microarchitectures," in *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2024, pp. 1–6.

[7] J. Zhang, C. Chen, J. Cui, and K. Li, "Timing side-channel attacks and countermeasures in cpu microarchitectures," *ACM Computing Surveys*, vol. 56, no. 7, pp. 1–40, 2024.

[8] A. Ahmad, K. Kim, M. I. Sarfaraz, and B. Lee, "Obliviate: A data oblivious filesystem for intel sgx." in *NDSS*, 2018.

[9] M. Schwarz, M. Lipp, C. A. Canella, R. Schilling, F. Kargl, and D. Gruss, "Context: A generic approach for mitigating spectre," in *Network and Distributed System Security Symposium 2020*, 2020.

[10] M. Taram, A. Venkat, and D. Tullsen, "Mitigating speculative execution attacks via context-sensitive fencing," *IEEE Design & Test*, 2022.

[11] L. Gerlach, D. Weber, R. Zhang, and M. Schwarz, "A Security RISC: Microarchitectural Attacks on Hardware RISC-V CPUs Artifact Repository," 2023. [Online]. Available: https://github.com/cispa/Security-RISC

[12] M. Sabbagh, Y. Fei, and D. Kaeli, "Secure speculative execution via risc-v open hardware design," June 2021.

[13] C. Celio, D. A. Patterson, and K. Asanovic, "The berkeley out-of-order machine (boom): An industry-competitive, synthesizable, parameterized risc-v processor," *EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2015-167*, 2015.

[14] A. Waterman, K. Asanovic, and J. Hauser, "The risc-v instruction set manual volume ii: Privileged architecture version 1.12," *EECS Department, University of California, Berkeley*, 2021.

[15] S. H. W. Cheng, C. Chuengsatiansup, D. Genkin, D. McNeil, T. Murray, Y. Yarom, and Z. Zhang, "Evict+ spec+ time: Exploiting out-of-order execution to improve cache-timing attacks," *Cryptology ePrint Archive*, 2024.

[16] H. Naghibijouybari, K. N. Khasawneh, and N. Abu-Ghazaleh, "Constructing and characterizing covert channels on gpgpus," in *Proceedings of the 50th annual IEEE/ACM international symposium on microarchitecture*, 2017, pp. 354–366.

[17] D. Evtyushkin, R. Riley, N. C. Abu-Ghazaleh, ECE, and D. Ponomarev, "Branchscope: A new side-channel attack on directional branch predictor," *ACM SIGPLAN Notices*, vol. 53, no. 2, pp. 693–707, 2018.

[18] J. J. Hoch and A. Shamir, "Fault analysis of stream ciphers," in *Cryptographic Hardware and Embedded Systems-CHES 2004: 6th International Workshop Cambridge, MA, USA, August 11-13, 2004. Proceedings 6*. Springer, 2004, pp. 240–253.

[19] Z. Wang and R. B. Lee, "Covert and side channels due to processor architecture," in *2006 22nd Annual Computer Security Applications Conference (ACSAC'06)*. IEEE, 2006, pp. 473–482.

[20] P. Kocher, "Differential power analysis," in *Proc. Advances in Cryptology (CRYPTO'99)*, 1999.

[21] N. Nazari, C. Fang, H. Mohammadi Makrani, B. Omidi, M. Eslamimehr, S. Rafatirad, A. Sasan, H. Sayadi, K. N. Khasawneh, and H. Homayoun, "Architectural whispers: Robust machine learning models fingerprinting via frequency throttling side-channels," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.

[22] K. Gandolfi, C. Mourtel, and F. Olivier, "Electromagnetic analysis: Concrete results," in *Cryptographic Hardware and Embedded Systems—CHES 2001: Third International Workshop Paris, France, May 14–16, 2001 Proceedings 3*. Springer, 2001, pp. 251–261.

[23] A. Rane, C. Lin, and M. Tiwari, "Raccoon: Closing digital {Side-Channels} through obfuscated execution," in *24th USENIX Security Symposium (USENIX Security 15)*, 2015, pp. 431–446.

[24] O. Oleksenko, B. Trach, T. Reiher, M. Silberstein, and C. Fetzer, "You shall not bypass: Employing data dependencies to prevent bounds check bypass," *arXiv preprint arXiv:1805.08506*, 2018.

[25] M. F. A. Kadir, J. K. Wong, F. Ab Wahab, A. F. A. A. Bharun, M. A. Mohamed, and A. H. Zakaria, "Retpoline technique for mitigating spectre attack," in *2019 6th International Conference on Electrical and Electronics Engineering (ICEEE)*. IEEE, 2019, pp. 96–101.

[26] J. Yu, L. Hsiung, M. El Hajj, and C. W. Fletcher, "Data oblivious isa extensions for side channel-resistant and high performance computing," *Cryptology ePrint Archive*, 2018.

[27] M. Sabbagh and Y. Fei, "Secure speculative execution via risc-v open hardware design," in *Fifth Workshop on Computer Architecture Research with RISC-V (CARRV 2021)*, 2021.

[28] A. Bhattacharyya, A. Sandulescu, M. Neugschwandtner, A. Sorniotti, B. Falsafi, M. Payer, and A. Kurmus, "Smotherspectre: exploiting speculative execution through port contention," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, 2019, pp. 785–800.

[29] O. Mutlu and J. S. Kim, "Rowhammer: A retrospective," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 8, pp. 1555–1571, 2019.

[30] A. G. Yağlikçi, M. Patel, J. S. Kim, R. Azizi, A. Olgun, L. Orosa, H. Hassan, J. Park, K. Kanellopoulos, T. Shahroodi *et al.*, "Blockhammer: Preventing rowhammer at low cost by blacklisting rapidly-accessed dram rows," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 345–358.